

A COMPLETE, PLAIN-LANGUAGE WALKTHROUGH

AQ-Mario: teaching a computer to imagine a video game

We trained four artificial neural networks to predict what happens next in a platform video game, measured what they actually learned inside, discovered that three of them were quietly broken in a way nobody had checked for, and built an automatic test that catches it. Every number in this document was measured, not estimated.

Experiment run 9–10 September 2026 · 4 training runs · 164,640 training steps total
~10 GPU-hours on NVIDIA H100 hardware · 8,000 gameplay episodes · 4.88 million frames
Repository: `/Users/sachin/aq-mario`

What is in this document

- | | |
|--|--|
| 1 The one-paragraph version | 10 The four experiments |
| 2 Every word explained | 11 Results |
| 3 Why a video game? | 12 Gates: automatic alarms |
| 4 What a world model is | 13 The discovery |
| 5 JEPA: predicting ideas, not pixels | 14 Playing the game |
| 6 The data: 8,000 lives | 15 What we learned |
| 7 Inside the machine | 16 All the numbers |
| 8 The cheating problem | 17 Links & further reading |
| 9 Probes: reading a mind | |

The one-paragraph version

Imagine teaching someone to play a video game by only ever letting them watch, never telling them any rules.

We showed a computer program eight thousand recorded playthroughs of a platform game level. For each moment it saw the screen and which buttons were held. We never told it where the character was, whether it was jumping, or whether it was about to die. Its only job was to **predict what the next few moments would look like**.

Then we opened it up and asked: did it figure out, on its own, where the character is on the screen? A previous project called **LeMario** had tried this and found the answer was mostly no — it scored **0.188** on a 0-to-1 scale for knowing the character's height. That is close to knowing nothing.

We reproduced that failure, found **why** it happens, fixed it with a well-known trick called a **momentum teacher**, and got **0.590** — more than three times better, still without telling the model anything. Adding cheap hints pushed it to **0.962**.

Then we found something nobody had checked

Every one of our models was ignoring the buttons. It had learned to predict the future from *momentum alone* — like guessing where a rolling ball goes without noticing anyone kicking it. All the existing tests passed. We built the test that catches it, and that test correctly predicted, in advance, which models could actually play the game.

Every word explained

Nothing here is assumed. Every abbreviation gets its full form and a plain-English meaning. Skim it now, come back when a word appears later.

The big ideas

JEPA

Joint Embedding Predictive Architecture. A design where a model predicts the future as a *summary* (an "embedding") rather than as actual pixels. Like predicting "she will be near the door" instead of painting the exact picture of her at the door. "Joint embedding" means both the past and the future get turned into summaries by the same summariser.

World model

A program that has learned how an environment behaves, so it can imagine "if I do *this*, what happens?" without actually trying it. Your own ability to predict that a dropped mug will break is a world model.

Latent / latent space / embedding

"Latent" means hidden. The model squashes a 224×224 colour picture (150,528 numbers) down to **192 numbers**. That list of 192 numbers is the latent, and the space of all such lists is latent space. Think of it as the model's private shorthand for "what is on screen right now".

Self-supervised

Learning without an answer key. Nobody labels the data. The model invents its own exercise — here, "predict the next moment" — and grades itself. This matters enormously for our claim: if the model learns where the character is *without being told*, that is a discovery. If we told it, it is just obedience.

Probe

A tiny separate model we train *afterwards* to read one fact out of the frozen latent. It answers: "is this information in there at all?" The big model is never changed by this — the probe is a thermometer, not a heater.

Measurements

R^2

Coefficient of determination. How much of something you can explain. **1.0** = perfect. **0.0** = no better than always guessing the average. **Negative** = worse than guessing the average, which is a real and common outcome and we report it honestly rather than hiding it at zero.

MSE

Mean Squared Error. Average of $(\text{guess} - \text{truth})^2$. Squaring punishes big misses much harder than small ones.

BCE

Binary Cross-Entropy. The scoring rule for yes/no questions, like "will this end badly within 5 steps?"

AUC & AP

Area Under the Curve and **Average Precision.** Two ways to score a rare-event detector. We use them because the event we care about happens in **0.96%** of frames — a detector that always says "fine" would be 99% accurate and completely useless, so plain accuracy is banned in this project.

Effective dimension (eff_dim)

Of the 192 numbers in the latent, how many are *actually doing work*? If the model cheats by making every screen look identical, this collapses toward 1. Healthy runs here sit around 28–35.

OOD

Out Of Distribution. Anything the model was never trained on. We trained only on one level, so every other level is OOD and results there mean something different.

Machinery

SIGReg

Sketched Isotropic Gaussian Regularisation. From the LeJEPa paper. A mathematical nudge that keeps the 192 numbers spread out in a nice round cloud instead of collapsing into a dot. "Isotropic" = the same in all directions; "Gaussian" = bell-curve shaped.

EMA

Exponential Moving Average. A slow-moving average that remembers the past. We keep a second copy of the model that lags behind the one being trained — the "momentum teacher". This single change was our biggest win.

Stop-gradient

Telling the model "you may look at this, but you may not change it." Used to stop a specific kind of cheating, at a cost we eventually measured.

ViT

Vision Transformer. A neural network that chops an image into a grid of small patches (here 14×14 pixels each) and lets every patch compare itself to every other patch. "Tiny" is the smallest standard size.

AdaLN-Zero

Adaptive Layer Normalisation, Zero-initialised. The mechanism by which button presses influence the prediction. "Zero-initialised" means it starts switched off and has to learn to switch itself on — which turns out to be central to this whole story.

BN

Batch Normalisation. Rescales numbers so they stay in a sensible range. It keeps running averages that go stale during training, which is why we refresh them before every measurement.

CEM

Cross-Entropy Method. A planning algorithm. Guess 256 random button sequences, imagine each one with the world model, keep the best 32, guess 256 more that look like those, repeat. It is evolution by natural selection, run four times, inside imagination.

PPO

Proximal Policy Optimisation. A standard reinforcement learning algorithm. We used it only to *generate* half the training data, never as part of the world model.

AdamW

Adam with decoupled Weight decay. The optimiser — the part that actually adjusts the model's millions of internal numbers.

bf16

Brain Floating Point, 16 bits. A compact number format that trades precision for range. We use it instead of the more common fp16 because SIGReg subtracts two nearly-equal numbers, and fp16 loses almost all its accuracy doing that.

Epoch / step / batch

A **batch** is 96 examples looked at together. A **step** is one adjustment of the model after one batch. An **epoch** is one full pass through all the data. Our runs: 3 epochs, 41,160 steps.

Infrastructure

GPU, H100, A10G

Graphics Processing Unit. Chips that do thousands of multiplications at once. **H100** is a fast expensive one (training); **A10G** is a slower cheap one (measuring).

Modal

A service that rents GPUs by the second and runs your code on them.

RAM (of the console)

Random Access Memory. The emulated console's memory. We read the true character position straight out of it — this is our ground truth, used *only* for grading, never for training.

Checkpoint

A saved snapshot of the model mid-training. We save ten per run, one every 10%.

Why a video game?

Because it is the rare case where we can check the model's homework perfectly.

The hard part of studying what a neural network "understands" is that you usually cannot verify it. If a model watches videos of city streets, and you want to know whether it has internally represented "how far away that car is", there is no way to get the true answer for every frame.

An emulated games console is different. The console's memory holds the exact character position as a number. We can read it for **every single frame**, perfectly, for free. So we can ask "does the model know where the character is?" and get an unarguable answer.

The rule we never broke

Those true positions are used **only to grade** the model, never to train it. The moment you train on them, a good score stops meaning "it figured this out" and starts meaning "we told it". One of our four experiments deliberately breaks this rule, and we report it separately for exactly that reason.

We used one level of a classic side-scrolling platform game, running in an emulator. The character runs right, jumps over gaps and hazards, and the camera follows. The goal post is **3,161 pixels** from the start.

What a world model is

A world model answers hypothetical questions about the future without having to live through them.

Before you cross a road, you do not step out and find out. You *imagine* stepping out, imagine the car arriving, and decide not to. That imagining machine is a world model. It takes **where things are now** plus **what you might do** and produces **what would happen**.

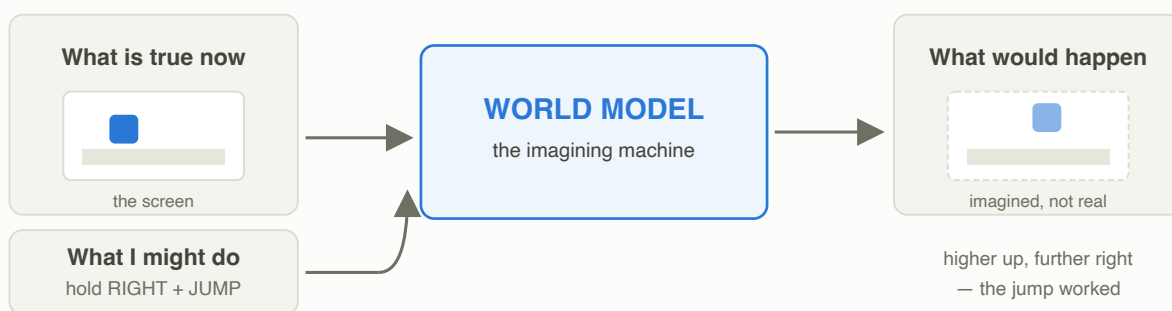


Figure 1. The shape of every world model. The blue square is a stand-in for the player character. Notice the third box is dashed: it never happened. That is the entire value — you get to find out without paying the price.

If a world model is accurate, you can chain it: imagine ten button presses ahead, try a thousand different sequences, and pick the one that ends up best. That is planning, and it is what we attempt in Section 14.

JEPA: predicting ideas, not pixels

The obvious way to build this is to predict the next picture. The obvious way is a trap.

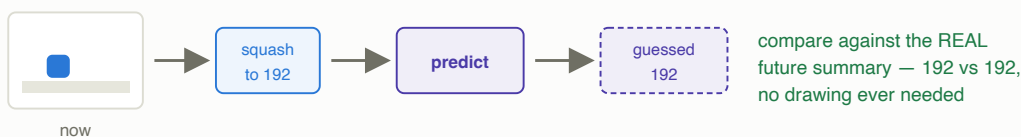
If you ask a model to draw the next frame exactly, it must get every cloud, every brick texture, every flicker right. Almost all of that effort goes into details that do not matter for deciding whether to jump. Worse, the parts that *do* matter — where the character is — are a tiny fraction of the pixels, so getting them wrong barely hurts the score.

JEPA (Joint Embedding Predictive Architecture) changes the target. Both the past and the future get squashed into 192-number summaries first, and the prediction happens *between summaries*. The model never draws anything.

THE TRAP — predict the picture



JEPA — predict the summary



784× fewer numbers to get right, and none of them are clouds.

Figure 2. Pixel prediction versus JEPA. 150,528 numbers is $224 \times 224 \times 3$ (width × height × red-green-blue). The summary is 192 numbers. The ratio is 784 to 1.

But this creates a brand-new way to cheat

If the model gets to invent the summaries *and* is scored on predicting them, there is a perfect trick: make every summary identical. Then prediction is trivially perfect and the score is flawless. The model has learned nothing and looks like a genius. This is called **collapse**, and Section 8 is about the arms race against it.

The data: 8,000 recorded lives

You cannot learn how a world works from a handful of examples. We generated 4.88 million.

What	Amount	Meaning in plain words
Episodes	8,000	complete playthroughs, start to death or timeout
Observations	4,880,000	individual moments recorded
Average length	669	moments per episode (measured, not estimated)
Storage	27 GB	compressed; 1,065 GB if stored raw
Frame size	224×224	pixels, colour

Two kinds of player, on purpose

Half the episodes came from a **random** player — buttons mashed at random. That sounds useless but is essential: a random player dies in every possible way, so the model sees plenty of hazards. Its weakness is that it rarely gets far, topping out around pixel 1,416 of 3,161.

The other half came from a **PPO** (Proximal Policy Optimisation) agent — a reinforcement-learning bot trained to make progress, reaching pixel 2,424 on average. It covers the later parts of the level the random player never sees.

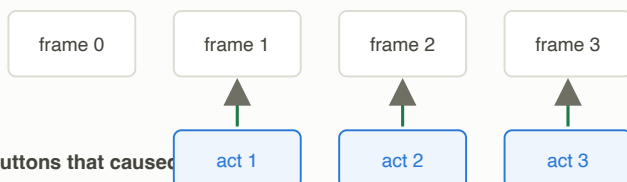
The PPO agent is not part of the world model

It is a camera operator, not an actor. It exists purely to record footage of the level's far half. The world model never sees its rewards, its goals or its decisions — only the resulting screens and buttons.

The off-by-one that would have ruined everything

In the recording, `actions[i]` is the button press that *caused* `frames[i]`. If you line these up wrong by a single position, you are asking the model to predict the *past* from the future. Nothing crashes. The loss still goes down, because consecutive button presses are similar. You simply get a subtly wrong model and no warning at all.

frames (what the screen showed)



RIGHT: act 1 caused frame 1
WRONG: act 1 caused frame 0
— the loss still falls, so you
never find out you are wrong

Frame 0 has no arrow: nothing before it caused it. That gap is why the alignment is easy to get wrong.

Figure 3. Action alignment. We wrote an automated test that builds fake data where the correct alignment is solvable and the shifted one is not; the correct version reaches an error below 0.05 while the shifted version cannot get within 5× of it.

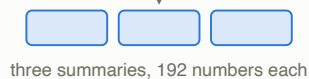
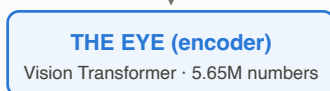
Ground truth we can trust

The character's position lives at specific addresses in the console's memory. Guessing the wrong address does not crash anything — it silently corrupts every measurement downstream. So the addresses are treated as *candidates* and verified empirically: the chosen pair reproduces the emulator's own reported position with $R^2 = 1.0000000000$ over 300 frames. The data collector refuses to run until this check has passed.

Inside the machine

Three parts: something that looks, something that listens to buttons, and something that imagines.

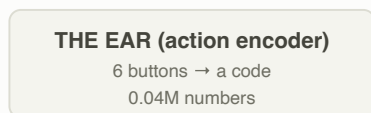
PAST — 3 moments the model is allowed to see



FUTURE — 5 guessed summaries



compared against the real
future summaries — that
difference is the whole lesson



Total trainable: 9.8 million numbers. A large modern language model has 100,000× more.

Figure 4. The architecture. Three real moments in, five imagined moments out, with button presses steering the imagination.

AdaLN-Zero: how buttons get a say

The buttons do not get glued onto the input. Instead they *modulate* the imagination at every layer — turning dials rather than adding ingredients. The "Zero" part means every dial starts at zero, so at the very first training step the buttons have **no effect at all**, and the model must learn to turn them up.

Remember this design choice

It was chosen specifically to stop the model ignoring the buttons. Section 13 is about discovering that it failed to do so, and that every test we had for it was checking the wrong thing.

The cheating problem

The model's best possible score is achieved by learning nothing. Everything in this section exists to remove that option.

Recall the trap: the model invents the summaries *and* is graded on predicting them. So the winning strategy is to make every summary the same. Prediction error goes to zero. The model is now a very expensive way of outputting one fixed list of 192 numbers.

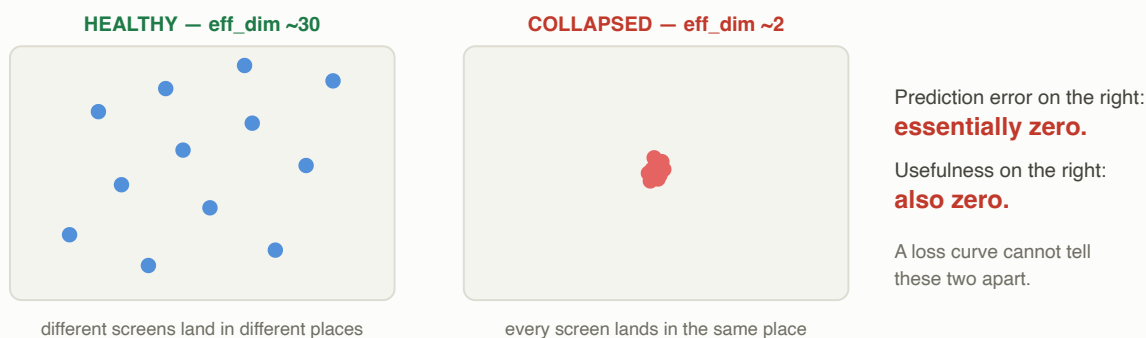


Figure 5. Collapse, drawn in 2 dimensions instead of 192. We measured a real collapsed run at effective dimension **2.0** while its prediction score read a magnificent-looking 0.97.

Defence one: SIGReg

SIGReg (Sketched Isotropic Gaussian Regularisation) adds a penalty that grows whenever the cloud of summaries stops being a nice round spread. It pushes back against clumping. Its strength is set by a number called lambda (λ).

We measured what different strengths do. Weak settings lose the fight outright:

λ (SIGReg strength)	Effective dimension	Verdict
0.1	1.08	collapsed
1.0	1.89	collapsed
10.0	10.3 → 21.1	holds, and rises

Defence two: the stop-gradient

SIGReg alone was not enough. The arithmetic explains why: collapsing is worth about **0.80** of prediction loss to the model, while the regularisation penalty costs it far less. Cheating simply pays better.

So we added a **stop-gradient**: the model may look at the true future summary but may not adjust itself to make that summary easier to predict. It has to come to the future, not drag the

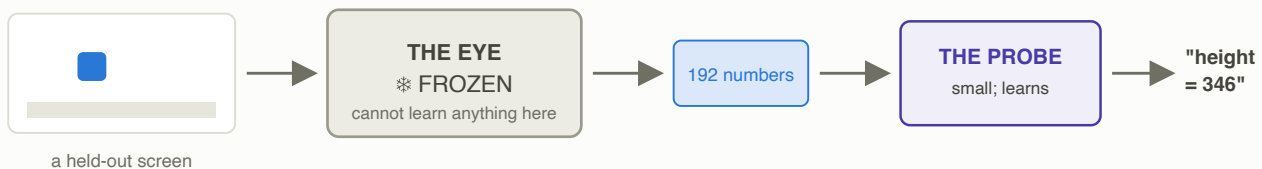
future toward it.

This works, and it has a hidden cost

The stop-gradient means the future frames contribute **no learning signal at all** to the eye. Whatever the model represents about the thing it is predicting is shaped only indirectly. In Section 10 this turns out to be the single most important fact in the entire experiment.

Probes: how you read a model's mind

Freeze the model. Ask a tiny student to read one fact out of its summaries. How well the student does tells you whether the fact was in there.



Score it against the console's true value. Good score → the information was present. Bad score → it was thrown away. Because the eye is frozen, the probe can only find what is already there. It cannot teach the eye anything.

Figure 6. Probing. The snowflake means the big model's numbers are locked. Only the small probe learns.

Reading R^2 without a statistics degree

R^2	What it means	In this project
1.00	perfect — every guess exactly right	never happens
0.96	excellent	our best height score
0.59	solid — most of the pattern captured	our best score with zero hints
0.19	weak — a faint trace	the previous project's score
0.00	useless — same as always guessing the average	—
-0.46	worse than useless	the broken model

The split protocol, and why it decides whether numbers mean anything

Consecutive frames of a video game are nearly identical. If you split them randomly, a probe can memorise frame 400 and then "predict" frame 401 perfectly — the score is meaningless. So we split by **complete playthrough**: 60 entire episodes are set aside and never used for training the probe. Same seed every time, so all four experiments are graded on the identical 60 episodes, 24,313 moments.

The four experiments

Same data. Same seed. Same 41,160 steps. One thing changed at a time.

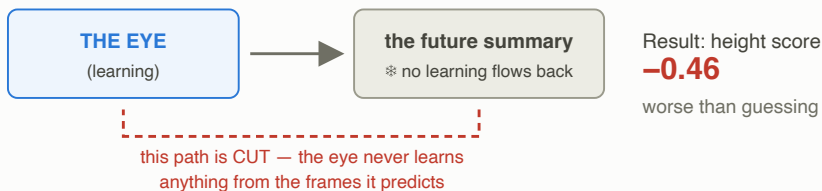
Every run used identical settings except the one variable being tested, so any difference in the results is attributable to that variable and nothing else.

Run	Target branch	Extra hints?	The question it answers
pure + stop-grad	stop-gradient	none	Does a plain self-supervised JEPA discover the character's height?
aux	stop-gradient	told height, camera, danger	If we simply tell it, how good does it get?
pure + EMA	momentum teacher	none	Is the stop-gradient the thing that was breaking it?
pure + EMA + inverse dynamics	momentum teacher	none	Can we force it to pay attention to the buttons?

The key idea: stop-gradient versus momentum teacher

Both stop the model from cheating, but they pay for it differently.

STOP-GRADIENT — the target is frozen solid



MOMENTUM TEACHER (EMA) — the target is a slow copy

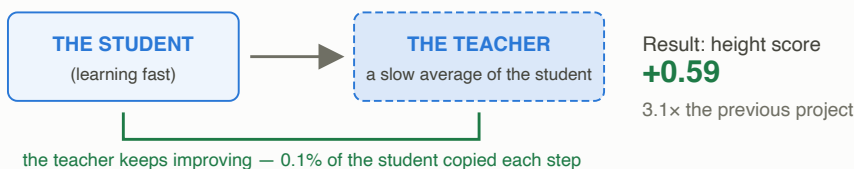


Figure 7. The single change that mattered most. A momentum teacher still cannot be chased (it moves too slowly to game in one step) but it keeps developing, so what the model represents about the future keeps improving too.

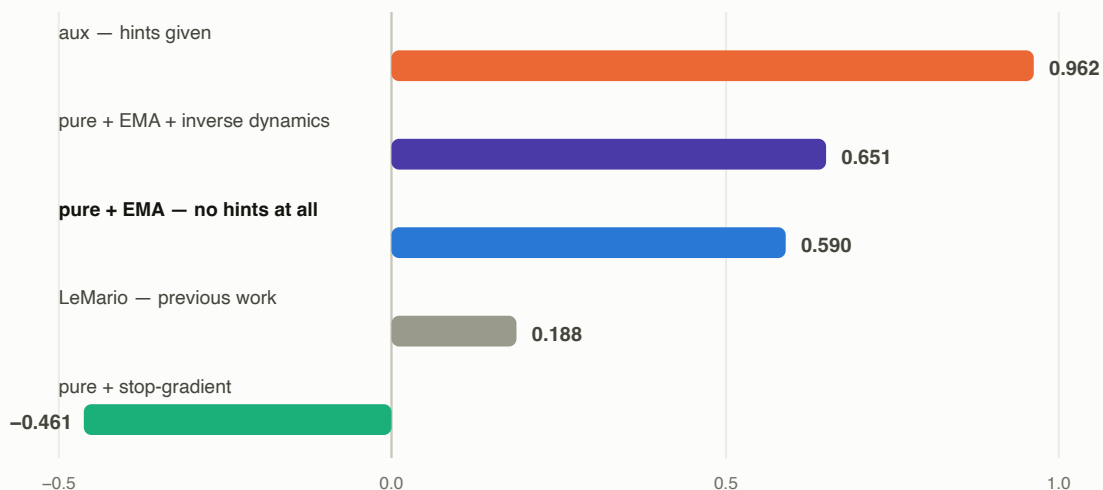
Why "momentum"?

Each training step, the teacher copies **0.1%** of the student and keeps 99.9% of itself. Over a thousand steps it drifts to roughly where the student was. Too fast and it becomes the student, and the cheating returns. Too slow and it stays a frozen random network. We used 0.999 and checked the effective dimension never collapsed — it reached **35.4**, the highest of any run.

Results

All four graded on the same 60 held-out playthroughs, 24,313 moments, after the full 41,160 steps.

Height (y) probe R^2 — higher is better

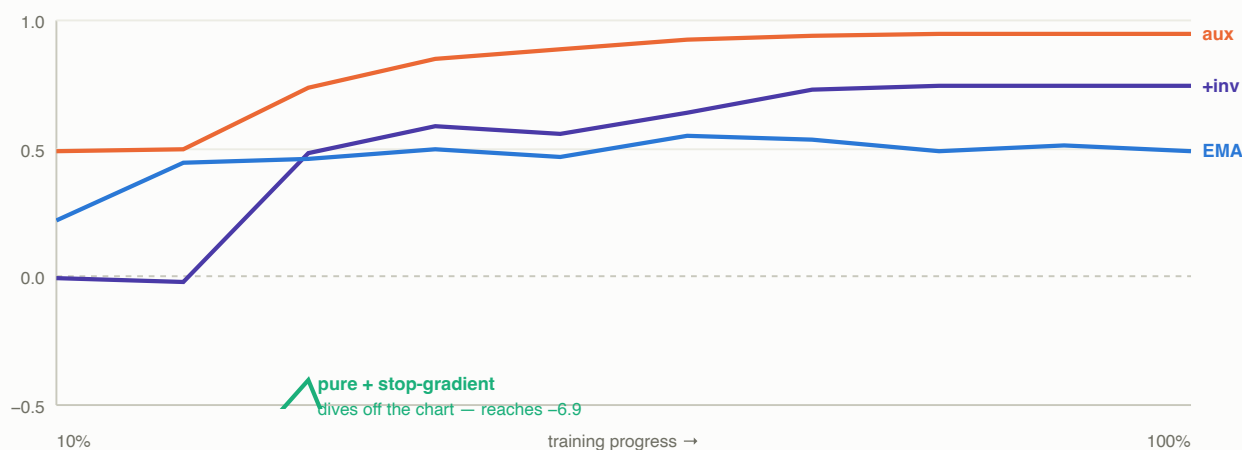


Grey = the score this project set out to beat. The bar left of the 0.0 line is worse than always guessing the average.

Figure 8. The headline. Only the orange bar was given hints; the blue and violet bars learned everything from watching.

How each run developed over time

Height probe R^2 , measured every 10% of training



The green line leaves the plot area almost immediately and never returns. It is shown clipped rather than rescaled, because rescaling to fit it would flatten the three lines that matter into a single band.

Figure 9. Measured at ten checkpoints per run. The orange line climbs steadily because it is being told the answer. The blue and violet lines climb because they are working it out.

The finding the previous work could not settle

The obvious excuse for a low self-supervised score is "it just needs longer." We tested that directly. At 4% of training the plain model scored -0.383 . After **ten times** more training it scored **-0.461** — it got *worse*. More training is not the answer; the stop-gradient was the problem.

Gates: automatic alarms

The point of the project: catch a broken model early, from the inside, without waiting for it to fail at a real task.

The previous project found its problem the expensive way — built a planner, the planner failed, then months later a hand-run probe revealed why. A **gate** inverts that: run the probes automatically at every checkpoint, and if the model has thrown away something essential, stop the run.

Our runs save a checkpoint and run the probes every 10%. Four health questions are asked each time, each matching a known way this can fail:

Check	Question	The failure it catches
learning	Is prediction error still falling?	training has stalled
not collapsing	Is effective dimension holding?	the cheat from Section 8
regularised	Is SIGReg in range?	the cloud is deforming
representing	Is the height probe rising?	the state is being discarded

Only the last one cannot be faked by a confident, collapsed model — which is why it is the one wired to actually stop the run.

It fired, on real data, exactly as designed

At 40% of the plain run, the height score had gone $-0.40 \rightarrow -1.60$ while prediction error read a perfectly healthy 0.2550 and the 5-step prediction score read **+0.745**. On the loss curve alone, nothing was wrong. The gate would have killed that run about an hour in. We let it continue deliberately, to see the whole trajectory — and it went on to get five times worse. The abort rule was right.

The rule needs *two* consecutive non-improving readings below the floor, never one. A single low reading is not evidence: the healthy model measured 0.12 at 10% simply because it was young. Killing on one number would have thrown away a run that was on its way up.

The discovery: the models were ignoring the buttons

Every probe passed. Every test passed. And the world model was not a world model.

A world model has to answer "what if I press *this*?" So we asked the simplest possible version of that question: hold RIGHT for eight moves, then hold LEFT for eight moves from the same starting point, and see how differently the model imagines them.

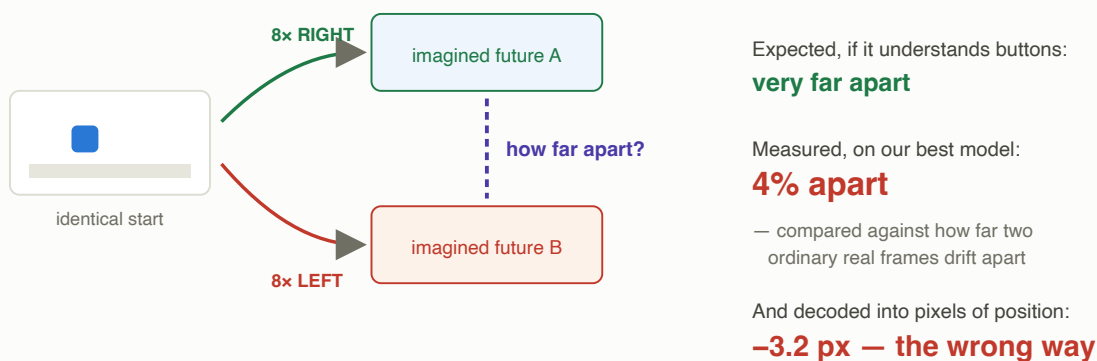


Figure 10. The test nobody had run. Holding RIGHT for eight moves produced an imagined future 4% different from holding LEFT — and when decoded into a position, RIGHT came out slightly *behind* LEFT.

Why every existing test missed it

There were already three tests guarding this, and all three passed:

1. AdaLN-Zero really is switched off at the start — **passes**
2. The predictor *can* respond when you change the button code — **passes**
3. The button pathway starts receiving learning signal at step 1 — **passes**

All three ask whether the machinery is **capable** of using buttons. None asks whether the finished model **does**. That gap is the entire finding, and it is an easy gap to leave in any system of this kind.

What "ignoring the buttons" actually looks like

The model predicts the future from **momentum**. Given three frames it can see the character is moving right at a certain speed, and simply continues that. For short horizons this is accurate enough to score well — which is precisely why the prediction loss never complained. It is a very good video predictor and a very poor world model.

The fix we tried, and why it failed instructively

We added **inverse dynamics**: a small extra network that must guess *which buttons were pressed* by looking at two consecutive summaries. The logic is that if the buttons must be recoverable, the summaries have to encode them.

It worked perfectly by its own measure — the button-guessing error fell from 0.695 to **0.0023**, essentially perfect. And the model got *worse* at control: its decoded position gap went from -3.2 px to **-67.4 px**.

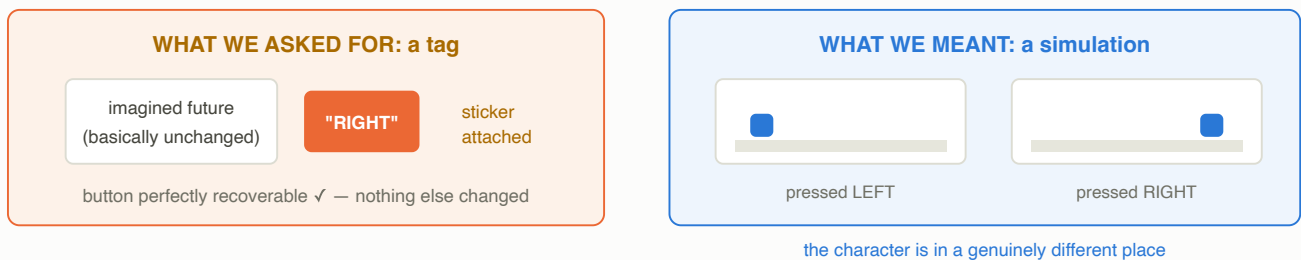


Figure 11. The lesson. "Make the action recoverable" and "make the action have an effect" are different objectives, and optimising the first can actively damage the second. A sticker satisfies the first perfectly.

Playing the game

With a world model you can plan: imagine many button sequences, keep the best, act. Here is how well that works when your action signal is 6 pixels wide.

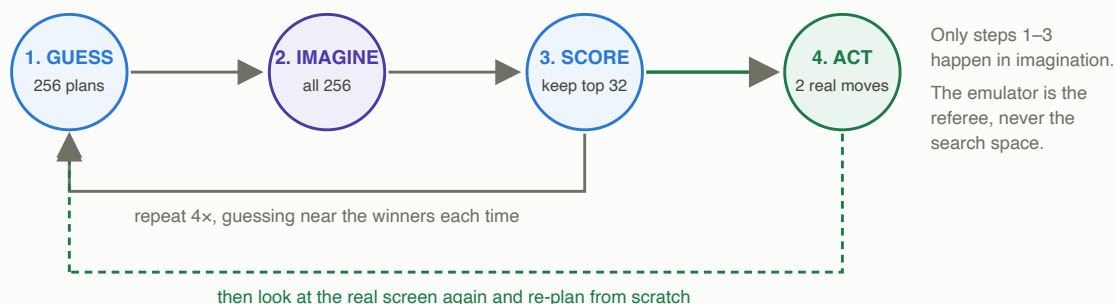


Figure 12. CEM (Cross-Entropy Method) planning. Evolution by natural selection, run four times per decision, entirely inside the model's imagination.

The result that validates the whole project

We measured each model's action gap **before** running any planner, then ran the planner. The gap predicted the outcome, in order, every time.

How far the planner actually got on World 1-1 (pixels)

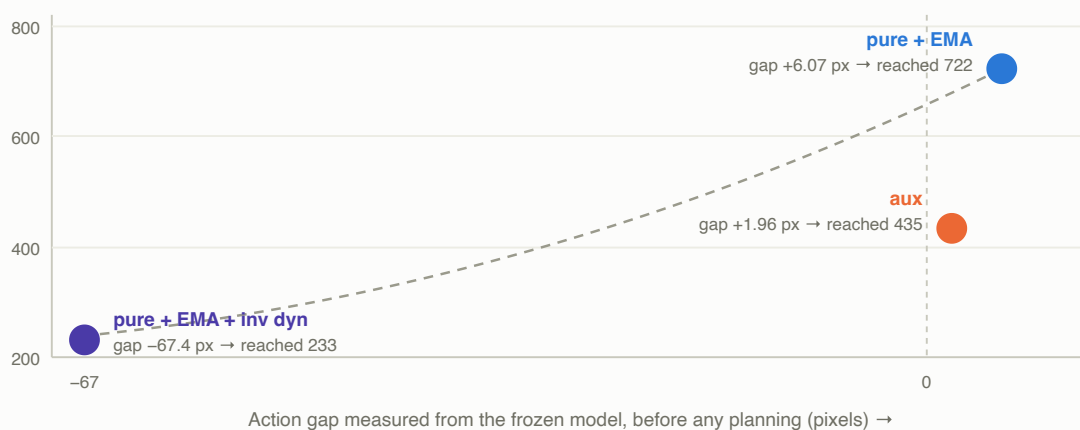


Figure 13. The thesis, demonstrated. A measurement taken from the frozen representation correctly ranked how well each model could control the game before any control was attempted.

How far did it get?

The best run reached **722 of 3,161 pixels — 23% of the level**. That is a planner that moves, not one that plays. Two things stop it, and both are measured rather than guessed:

- **The action signal is +6 pixels** when the true effect of holding right versus left is roughly 200. The planner is steering on about 3% of the real signal.
- **The danger detector reads 0.000** on imagined futures, so the planner has *no working hazard avoidance at all*. Given a longer time limit, two of three runs found a way to die.

We also tested whether the planner itself was at fault by trying six different planner settings — shorter plans, penalties for changing direction, restored danger ranking, and double the search budget. **None beat the original.** Doubling the search made it *worse*, which is the classic signature of an optimiser finding more of a model's mistakes rather than more of the task's solutions. The bottleneck is the model, not the search.

On the other levels

We tested four levels the model had never seen. It failed on all of them — typically dying within 45–140 moves. This is expected and is reported as a negative result: every one of the 8,000 training episodes came from a single level, and the probes that score the planner were fitted on that level too, so two different things fail at once and the design cannot separate them.

What we learned

Claimable

1. **A momentum teacher fixes self-supervised JEPA on this task.** Height score $-0.461 \rightarrow +0.590$ from one change, with no labels of any kind. That is $3.1 \times$ the previous published result.
2. **More training does not rescue the broken configuration.** Ten times the training made it slightly worse, which settles the obvious objection.
3. **A representation-level measurement predicted control ability in advance**, correctly ordering three models before any planner ran.
4. **Action-recoverability and action-effect are different objectives.** Optimising the first to near-perfection made the second substantially worse.

Not claimable, and worth saying plainly

- The 0.962 score came from a model that was **told** the height. It measures optimisation working, not discovery.
- On horizontal position the previous work still scores better (0.997 against our 0.830). We win decisively on height and lose on that.
- 23% of one level is not "playing the game", and nothing here transfers to other levels.
- Every number is from a single seed. Nothing here has error bars.

What we would do next

Put the loss directly on the quantity that matters: train the model so that imagining "hold right" genuinely ends further right than imagining "hold left". That is exactly what the gate measures and exactly what the planner consumes — so for the first time the training objective, the alarm and the task would all be optimising the same thing. A sticker cannot satisfy it.

All the numbers

Every figure below is measured. Final gate: 60 held-out complete playthroughs, 24,313 moments, identical split across all four runs.

Final scores

Model	height (y)	position (x)	camera	danger AUC	action gap
aux (hints given)	0.9620	0.8303	0.8381	0.9838	+1.96 px
pure + EMA + inv dyn	0.6511	0.7541	0.7577	0.8730	−67.36 px
pure + EMA	0.5904	0.6883	0.6938	0.8556	+6.07 px
pure + stop-grad	−0.4613	−0.1309	−0.1274	0.6523	not measurable
LeMario (previous work)	0.188	0.997	—	—	—

Height score at every checkpoint (the data behind Figure 9)

Progress	aux	pure+EMA	+inv dyn	pure+stop-grad
10%	0.492	0.218	−0.007	−1.147
20%	0.499	0.449	−0.022	−0.948
30%	0.738	0.460	0.486	−0.403
40%	0.849	0.497	0.589	−1.597
50%	0.885	0.471	0.556	−5.984
60%	0.924	0.550	0.638	−6.877
70%	0.939	0.535	0.734	−3.571
80%	0.947	0.491	0.748	−1.945
90%	0.948	0.516	0.747	−3.042
100%	0.949	0.493	0.746	−2.623

These are the cheap in-training probe (24 playthroughs). The final full-protocol scores in the table above (180 playthroughs) are the ones to quote.

Planner results, World 1-1

Model	best reached	% of level	note
pure + EMA, longer limit	722	23%	best overall
pure + EMA	560	18%	400-move limit

aux	435	14%	—
pure + EMA + inv dyn	233	7%	inverted action signal

Training configuration

Setting	Value	Why
Steps / epochs	41,160 / 3	10.3× the previous checkpoint
Batch size	96	256 exhausts a 24 GB card at this window length
Learning rate	3×10^{-4} , cosine	standard
Precision	bf16	fp16 destroys the SIGReg subtraction
SIGReg λ	10.0	0.1 and 1.0 both collapse (measured)
Latent size	192	the summary length
History / horizon	3 / 5	moments seen / moments predicted
Trainable parameters	9.8M	plus 0.1M for the inverse-dynamics head
Hardware	1× H100 per run	~2.5 h each

Links & further reading

Where each idea in this document comes from, roughly easiest first.

Start here if you want the intuition

What	Where
World Models — Ha & Schmidhuber. The paper that made "let the agent dream" a mainstream idea. Very readable.	arxiv.org/abs/1803.10122
I-JEPA — the image version of the architecture used here, and the clearest explanation of why predicting summaries beats predicting pixels.	arxiv.org/abs/2301.08243
V-JEPA — the video version, closest in spirit to what we built.	arxiv.org/abs/2404.08471

The specific methods we used

What	Where
LeJEPA — the source of SIGReg, the anti-collapse penalty. A copy sits in the repository at <code>paper/LeJEPA-2511.08544.pdf</code> .	arxiv.org/abs/2511.08544
BYOL — where the momentum (EMA) teacher comes from. This is the idea that produced our biggest single improvement.	arxiv.org/abs/2006.07733
Vision Transformer (ViT) — the "eye".	arxiv.org/abs/2010.11929
DiT — introduced adaLN-Zero, the mechanism by which button presses modulate the prediction.	arxiv.org/abs/2212.09748
PETS — planning with a learned model using the Cross-Entropy Method, which is what Section 14 does.	arxiv.org/abs/1805.12114
PPO — the algorithm behind the bot that recorded half our data.	arxiv.org/abs/1707.06347
AdamW — the optimiser.	arxiv.org/abs/1711.05101

Software

What	Where
gym-super-mario-bros — the environment wrapper.	github.com/Kautenja/gym-super-mario-bros
nes-py — the emulator underneath it.	github.com/Kautenja/nes-py
timm — supplies the Vision Transformer.	github.com/huggingface/pytorch-image-models
PyTorch — the framework everything is written in.	pytorch.org/docs
Stable-Baselines3 — the PPO implementation.	stable-baselines3.readthedocs.io

Concepts worth looking up

Term	Where
Coefficient of determination (R^2)	en.wikipedia.org/wiki/Coefficient_of_determination
Marchenko–Pastur distribution — tells us what effective dimension a <i>random</i> model would show, so we know what number to beat	en.wikipedia.org/wiki/Marchenko–Pastur_distribution
Cross-entropy method	en.wikipedia.org/wiki/Cross-entropy_method
bfloat16 number format	en.wikipedia.org/wiki/Bfloat16_floating-point_format
Batch normalisation	arxiv.org/abs/1502.03167

The code in this project

File	What lives there
<code>aqmario/model.py</code>	the eye, the ear, the imagination, and the momentum teacher
<code>aqmario/losses.py</code>	SIGReg, the hint heads, inverse dynamics
<code>aqmario/train.py</code>	the training loop, checkpoints, the in-training alarm
<code>aqmario/gates.py</code>	all the measurements, including the action-conditioning gate
<code>aqmario/plan.py</code>	the CEM planner
<code>aqmario/data.py</code>	the recording format and the trajectory split
<code>modal_app.py</code>	everything that runs on rented GPUs
<code>tests/</code>	168 automated tests, including the ones that would have caught this
<code>media/</code>	the planner videos

One honest closing note

The most useful result in this document is a **negative** one: a fix that worked perfectly by its own measure ($0.695 \rightarrow 0.0023$) made the thing we actually cared about substantially worse. That only became visible because we had built a measurement of the thing we actually cared about. If there is one transferable lesson here, it is that the metric you optimise and the metric you need are rarely the same thing, and the gap between them is invisible until you measure it directly.